Protocol Execution: From Voter Ballot to Published Tally

Phase 1 — Setup (Trusted Third Party TP)

1. TP chooses safe primes p, $q \rightarrow N = p \cdot q$
2. TP picks $H \in Z^*_{N^2} \rightarrow$ publishes (N, H), goes offline
3. Aggregator A generates $sk_A \in Z^*_{N^2} \rightarrow$ publishes $pk_A = H^{sk_A} \bmod N^2$
4. Each voter U_i independently picks $sk_i \in [0, N^2)$ (no key dealer, TP does not know sk_i)

Assumptions: A1 ($N=pq$, odd primes), A2 ($sk_A \neq H$, $\gcd(sk_A,N)=1$)

Phase 2 — Voter U_i (Browser / WASM)

Step 1: Select candidate → vote vector

e.g., candidate 2 of 3 → $v = [0, 1, 0]$

Step 2: Pack into integer (25-bit slots)

$m = v_1 \cdot 2^{50} + v_2 \cdot 2^{25} + v_3 \cdot 2^0 = 2^{25}$

Step 3: Paillier encrypt (sk_i = randomness r_i)

$c_i = H^{sk_i} \cdot (1+N)^m \bmod N^2$

Step 4: Compute auxiliary value

$aux_i = pk_A^{sk_i} = H^{sk_A \cdot sk_i} \bmod N^2$

Assumptions: A4 ($k \cdot 25 < \log_2 N$), A5 ($v_{i,j} \in \{0,1\}$), A6 ($sk_i = r_i$)

c_i (to chain)

aux_i (to collector)

Phase 3 — Collector C

Receives all aux_i from voters

Computes product:

$aux = \Pi\, aux_i \bmod N^2$

$= H^{sk_A \cdot \Sigma sk_i} \bmod N^2$

Sends aux → Aggregator A

$\{c_i\}$

aux

Phase 4 — Aggregator A (Native C / libtommath)

Input: ciphertexts $\{c_1, \ldots, c_n\}$, aux, (N, H, sk_A)

Step 1. Product of ciphertexts

$\Pi = \Pi_{i=1}^{n} c_i \bmod N^2$

$= [\Pi (1 + x_i \cdot N)] \cdot H^{\Sigma sk_i} \bmod N^2$ (binomial expansion, Lemma 1)

$= (1 + S \cdot N) \cdot H^{\Sigma sk_i} \bmod N^2$ where $S = \Sigma x_i$

Step 2. Exponentiate by sk_A

$P = \Pi^{sk_A} \bmod N^2$

$= (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \Sigma sk_i} \bmod N^2$

Step 3. Cancel mask with aux

$P' = P \cdot aux^{-1} \bmod N^2$

$= (1 + S \cdot N)^{sk_A} \cdot H^{sk_A \cdot \Sigma sk_i} \cdot H^{-sk_A \cdot \Sigma sk_i} \bmod N^2$

$= (1 + S \cdot N)^{sk_A} \bmod N^2$ (Theorem 2: mask cancelled)

Step 4. Apply L-function

$L(P') = (P' - 1) / N = sk_A \cdot S \bmod N$ (binomial: $(1+SN)^{sk_A} \equiv 1 + sk_A \cdot S \cdot N \bmod N^2$)

Step 5. Multiply by sk_A^{-1} to recover sum

$R = L(P') \cdot sk_A^{-1} \bmod N = S \bmod N$ (Theorem 3)

Since $S < N$ (Assumption A3): $R = S$ exactly

Phase 5 — Unpack & Publish Results

$R = \Sigma x_i = \Sigma_{i=1}^{n} [v_{i,1} \cdot 2^{25(k-1)} + v_{i,2} \cdot 2^{25(k-2)} + \ldots + v_{i,k} \cdot 2^0]$

$= \Sigma_j [(\Sigma_i v_{i,j}) \cdot 2^{25(k-j)}]$ (slots are independent, Corollary 1)

Extract: $count_j = (R \gg 25(k-j)) \bmod 2^{25}$ for $j = 1..k$

Publish: $[count_1, count_2, \ldots, count_k]$ → Portal visualization + on-chain record

Assumptions for Correctness (referenced in proofs):

(A1) $N = p \cdot q$, p and q distinct odd primes → $Z^*_{N^2}$ well-defined, inverses exist

(A2) $sk_A \neq H$ and $\gcd(sk_A, N) = 1$ → non-degenerate key, $sk_A^{-1} \bmod N$ exists

(A3) $S = \Sigma x_i < N \rightarrow R = S \bmod N = S$ (exact, no wraparound)

(A4) $k \times 25 < \log_2 N$ → packed vote fits in single Paillier plaintext

(A5) $v_{i,j} \in \{0,1\}$ and $\Sigma_i v_{i,j} < 2^{25}$ → no per-slot overflow

(A6) $sk_i = r_i$ (key as randomness) → $aux_i = pk_A^{r_i}$ cancels H^{r_i} in aggregation

Privacy: Under DCR assumption, c_i and aux_i are indistinguishable from random in $Z^*_{N^2}$